

# Elastic *Stack*

Bilaketa, behagarritasuna eta log-en kudeaketa

**Zikloa** Adimen Artifiziala eta Big Data

**Modulua** Big Data Sistemak



# Elastic Stack

- 
- 01 Sarrera: log kaosa eta pipeline-a
  - 02 Elasticsearch: alderantzizko indizea eta datu-eredua
  - 03 Query DSL: kontsultak eta agregazioak
  - 04 Kibana: bistaratzea eta ILM
  - 05 Ingesta: Beats eta Logstash (grok)
  - 06 Ariketak eta ondorioak

# 01 Sarrera eta arkitektura

“

*Gaueko 3:00etan bezero batek deitu du: webgunea erori da. 50 zerbitzari daude. SSH, grep, SSH, grep... ordu bat igaro da. Oraindik ez dakigu non da arazoa.*

— ELASTIC STACK-EN ZERGATIA · LOG KAOSA

# Zer da Elastic Stack?

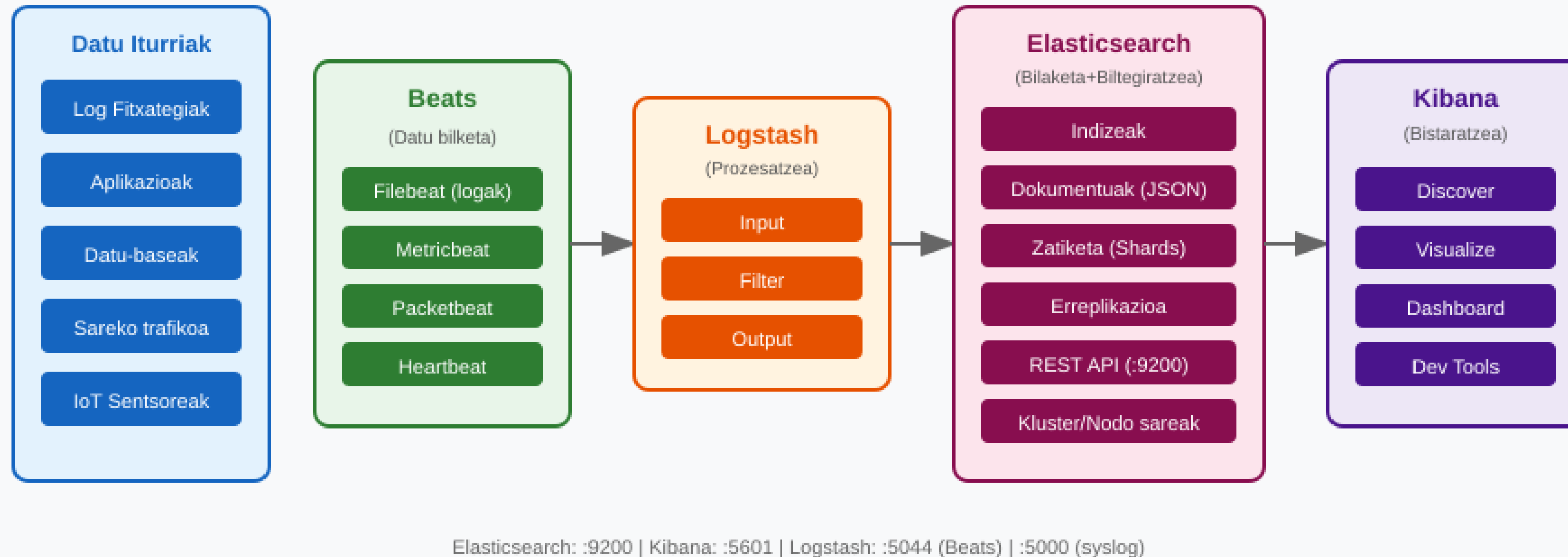
- Kode irekiko plataforma datu kopuru handiak **biltzeko, bilatzeko, aztertzeko eta bistaratzeko**
- Historikoki **ELK Stack: Elasticsearch + Logstash + Kibana**
- Beats gehitu ondoren, **Elastic Stack** izena hartu zuen
- Erabilera nagusiak: **bilaketa, behagarritasuna, segurtasuna, IoT eta IA**
- Ez da soilik log-ak kudeatzeko sistema bat



## — ARKITEKTURA

# Elastic Stack: Datu-Fluxua

**Iturria → Beats → [Logstash] → Elasticsearch → Kibana.** Logstash aukerakoa da (datuak egituratuta badaude, zuzenean ES-era).



# Hiru bloke nagusi

- **Observabilitatea** — logak, metrikak, APM
- **Segurtasuna** — SIEM, iruzur-detekzioa
- **Bilaketa** — e-commerce (eBay: 1.000M produktu)
- Gehiago: IoT, geospaziala, RAG/AI (2023+)

## Elastic Stack: Erabilera-kasuak

### A. Observabilitatea

4 erabilera-kasu

-  Log Analisi Zentralizatua
-  APM — Errendimendu Monitorizazioa
-  Sistema Monitorizazioa (CPU/RAM)
-  LLM / AI Observability

### B. Segurtasuna

3 erabilera-kasu

-  SIEM — Segurtasun Analisia
-  Iruzur Detekzioa
-  Zibersegurtasun Analitika

### C. Datu Gestio & AI

5 erabilera-kasu

-  E-commerce Bilaketa
-  Mantenu Prediktiboa (IoT)
-  Analisi Geospazialak
-  Osasungintza
-  RAG eta Vektore-bilaketa

# 4 Osagaiak · Rol Bakoitza

- **Elasticsearch** — Datuak gorde, indexatu eta bilatzeko motor banatua. Stack-aren **bihotza**. REST API: **:9200**
- **Logstash** — Datuak jaso, eraldatu eta bidaltzeko pipelinea: **INPUT → FILTER → OUTPUT**
- **Kibana** — Elasticsearch-eko datuak kontsultatu, aztertu eta bistaratzeko web interfazea. **:5601**
- **Beats** — Datu-iturburuetan instalatzen diren **agente arinak**, datuak jaso eta bidaltzeko.



— KONPARAKETA

# Noiz aukeratu Elastic?

## Elastic Stack vs Alternatiboa

|                  | Elastic Stack | Splunk         | Grafana+Loki | SQL DB   |
|------------------|---------------|----------------|--------------|----------|
| Log analisia     | ✓             | ✓              | ✓            | ⚠        |
| Testu bilaketa   | ✓             | ✓              | ✓            | ✗        |
| SIEM segurtasuna | ✓             | ✓              | ⚠            | ✗        |
| Mantenu predikt. | ✓             | ⚠              | ⚠            | ✗        |
| Geospazialak     | ✓             | ✓              | ⚠            | ✓        |
| RAG / AI         | ✓             | ⚠              | ⚠ ⚠          | ✗        |
| Kostua           | ● Kode irekia | ● Oso garestia | ● Doakoa     | ● Merkea |
| Erraztasuna      | ● Ertaina     | ● Erraza       | ● Erraza     | ● Erraza |

- **Splunk** — Antzeko erabilera-esparru zabala: logak, behagarritasuna eta segurtasuna. **Komertziala**, hainbat lizentzia/precio eredurekiin.
- **Grafana + Loki** — Logen bilketa, bilaketa eta bistaratzeko aukera indartsua; Loki-k batez ere metadata/labels indexatzen ditu eta kostu txikiagoa bilatzen du.
- **SQL DB** — Datu egituratuak eta **transakzioak (OLTP)** kudeatzeko aproposa; ez da Elastic-en ordezkoko zuzena testu-bilaketa masiborako.
- **Elastic Stack** — Bilaketa + log analisia + behagarritasuna + segurtasuna + bilaketa bektoriala/AI plataforma berean.

## — 1. PRAKTIKA

# ELASTICSEARCH + KIBANA MARTXAN JARRI

### HELBURUA:

Docker Compose erabiliz **Elasticsearch** eta **Kibana** abiarazi eta bi zerbitzuak erabilgarri daudela egiaztatu.

### PAUSOAK:

1. Sortu lanerako karpeta, **compose.yaml** fitxategiarekin
2. Ingurunea abiarazi
3. Probatu nabigatzailean:
  - Elasticsearch -> localhost:9200
  - Kibana → localhost:5601
4. Zer erakusten du Elasticsearch-ek localhost:9200 irekitzean?  
Zer desberdintasun dago Kibana irekitzean?

```
compose.yaml
1  services:
2
3      elasticsearch:
4          image: docker.elastic.co/elasticsearch/elasticsearch:9.5.4
5          container_name: iabd-elasticsearch
6          environment:
7              - discovery.type=single-node
8              - xpack.security.enabled=false
9          ports:
10             - "9200:9200"
11          volumes:
12             - elasticsearch_data:/usr/share/elasticsearch/data
13
14      kibana:
15          image: docker.elastic.co/kibana/kibana:9.5.4
16          container_name: iabd-kibana
17          ports:
18             - "5601:5601"
19          environment:
20             - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
21          depends_on:
22             - elasticsearch
23
24  volumes:
25      elasticsearch_data:
```

# 02 Elasticsearch

# Zergatik da Elasticsearch azkarra?

## SQL-ren arazoa:

- Datu kopuru handietan **garestia** izan daiteke.
- Ez dago testu-bilaketarako bereziki optiizatuta.
- Ez du berez **errelevantzia-puntuaziorik**.
- **LIKE**-ek ez ditu berez antezko hitzak edo akatsa ortografikoak kudeatzen.

## Elasticsearch:

- Dokumentuak sartzean **alderantzizko indizea** eraikitzen du.
- Bilaketa **indize horren gainean** egiten du.
- Datuak **JSON dokumentu** gisa kudeatzen ditu.
- **REST API** bidez kontsultatzen da.
- **\_score** → emaitzak errelevantziaren arabera ordenatzeko

## GAKOA: ALDERANTZIZKO INDIZEA

# Alderantzizko indizea

Liburu baten aurkibidea bezala: **hitza**  
→ **zein dokumentutan** dagoen.  
Bilaketa indizean egiten da, ez datuetan — millisegundotan.

## Elasticsearch: Alderantzizko Indizea

### Dokumentuak

Doc 1: "Elasticsearch bilaketa azkarra"  
id=1

Doc 2: "Kibana bistaratzea azkarra"  
id=2

Doc 3: "Logstash datuak bilaketa"  
id=3

Indexatu

### Alderantzizko Indizea

| Hitza (Term)  | Dokumentuak  |
|---------------|--------------|
| bilaketa      | Doc 1, Doc 3 |
| azkarra       | Doc 1, Doc 2 |
| elasticsearch | Doc 1        |
| kibana        | Doc 2        |
| bistaratzea   | Doc 2        |
| logstash      | Doc 3        |
| datuak        | Doc 3        |

Bilaketa "bilaketa" → Doc 1 eta Doc 3 berehala aurkitu

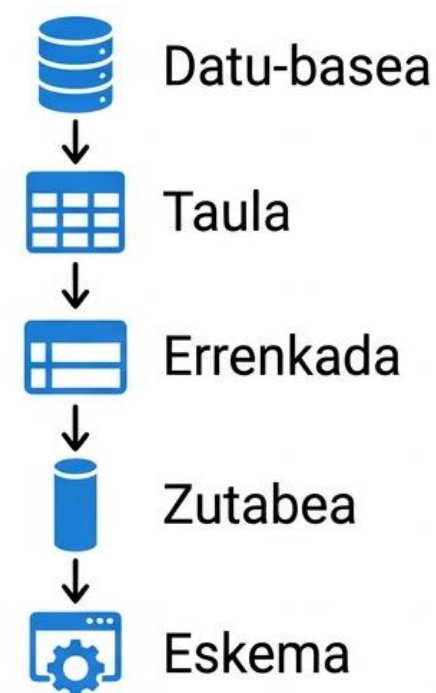


# Indizea, dokumentua, mapaketa

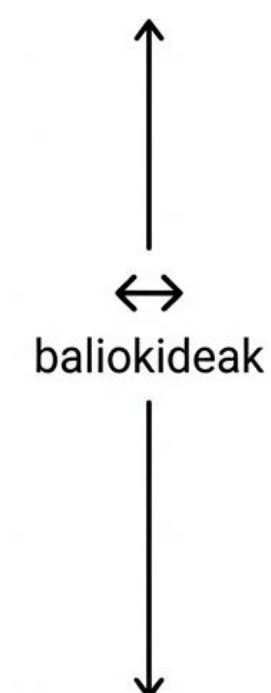
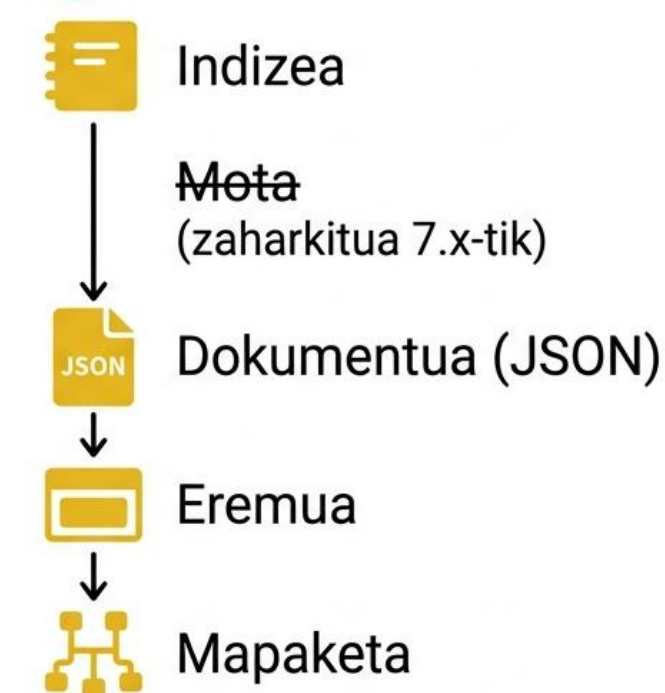
- Indizea ≈ taula
- Dokumentua ≈ errenkada → JSON
- Eremua ≈ zutabe
- Mapaketa (mapping) ≈ eskema
- ⚠ Ez da SQLko JOIN ereduan oinarritzen — denormalizatu (dena dokumentu berean)
- Datu erlazionatuak dokumentu berean gordetzea → bilaketa sinpleago eta azkarragoa

## Datu-eredua: Erlazionala vs Elasticsearch

### Datu-base Erlazionala



### Elasticsearch



⚠ ES-ek ez du JOIN operadorerik — harreman konplexuak aplikazio-mailan konpondu

# Elasticsearch: Eremu Motak

⚡ **text** vs **keyword**: akats ohikoenak!



## text

Testu osoko bilaketa · Tokenizatua ·  
match kontsultak · Ezin ordenatu  
Koaderno gorria arina



## keyword

Balio zehatza · Ez tokenizatua ·  
term kontsultak · Ordenatu/Agregatu ✓  
bulegoa



## integer / long

Zenbaki osoak · range kontsultak  
42



## float / double

Zenbaki hamartarrak  
2.99



## date

Data eta orduak · ISO8601 formatua ·  
now-1h espresioak  
2024-01-15



## boolean

Egia/Faltsua · Iragazketa sinplea  
true / false



## geo\_point

Lat/Lon kokapena · Gertuko bilaketa ·  
Geo-fencing  
{lat: 43.26, lon: -2.92}



## nested

Objektu-zerrenda · Harreman konplexuak  
[{...}, {...}]

## — EREMU-MOTAK

# text vs keyword

**text** — tokenizatua, `match`  
bilaketarako (testu librea).

**keyword** — bere horretan, `term`  
bilaketarako (iragazi, ordenatu,  
agregatu).

# Pitfall klasikoena

## *text vs keyword*

text eremu batean term kontsulta egiteak EZ du ezer aurkitzen: text tokenizatuta eta txikiz gordetzen da ('koadernoa'), baina term-ek balio zehatza bilatzen du ('Koadernoa').  
Konponbidea: izena.keyword.

### GARRANTZITSUA

Askotan eremu bera bietara nahi da → multi-field mapaketa: izena (text, bilatzeko) + izena.keyword (keyword, agregatzeko).



## — 2. PRAKTIKA

# LEHEN INDIZEA ETA DOKUMENTUAK

### HELBURUA:

Kibana **Dev Tools** erabiliz Elasticsearch-en indize bat sortu, JSON dokumentuak txertatu eta datuak kontsultatu.

### PAUSOAK:

1. Ireki Kibana → Dev Tools

2. Sortu lehen dokumentua:

```
1 POST produktuak/_doc
2 {
3   "izena": "Koaderno urdina",
4   "kategoria": "Papergintza",
5   "prezioa": 4.50,
6   "stock": 25
7 }
```

3. Gehitu beste 3 dokumentu:

```
POST produktuak/_doc
{
  "izena": "Koaderno handia",
  "kategoria": "Papergintza",
  "prezioa": 8.50,
  "stock": 10
}
```

```
POST produktuak/_doc
{
  "izena": "Sagu optikoa",
  "kategoria": "Informatika",
  "prezioa": 19.99,
  "stock": 15
}
```

```
POST produktuak/_doc
{
  "izena": "Teklatu mekanikoa",
  "kategoria": "Informatika",
  "prezioa": 59.99,
  "stock": 5
}
```

4. Kontsultatu indizeko dokumentuak:

```
GET produktuak/_search
```

5. Aztertu emaitza:

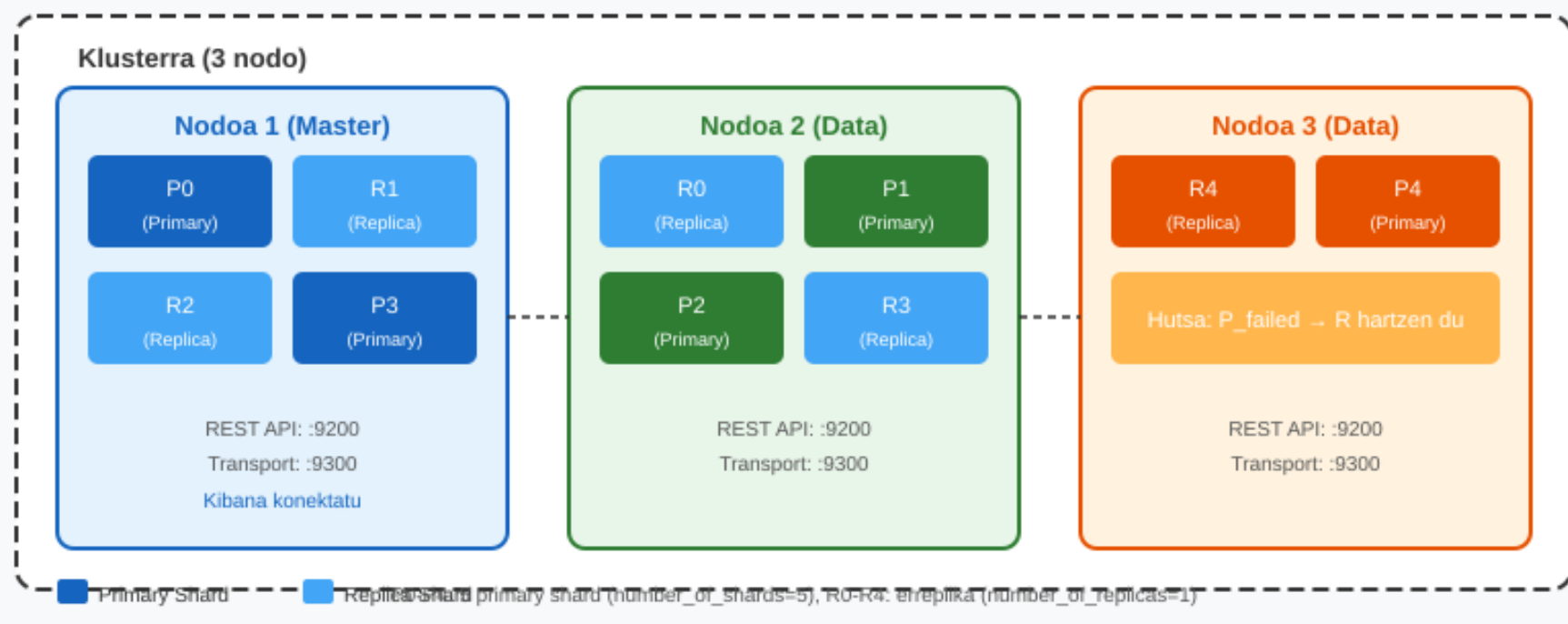
- Zenbat dokumentu daude?
- Non agertzen dira dokumentuaren benetako datuak?
- Zer da produktuak?
- Zer da { "izena": ..., "prezioa": ... } bakoitza?

## — ESKALA

# Shards eta replicas

- **Shard** — indizearen zati bat. Datuak nodoen artean **banatzeko** erabiltzen da → eskala horizontala
- **Replica** — shard baten **kopia**. Erabilgarritasuna eta bilaketa-errendimendua hobetzen ditu.
- ⚠ **Primary shard kopurua** indizea sortzean definitzen da; aldatzeko normalean **reindexatu** behar da. **Replica kopurua alda daiteke**.
- **Klusterraren osasuna:**
  - 🟢 GREEN – primary eta replica guztiak esleituta
  - 🟡 YELLOW – primary guztiak esleituta, baina replica batzuk ez
  - 🔴 RED – primary shard bat edo gehiago ez dago erabilgarri

### Elasticsearch: Zatiketa (Shards) eta Erreplikazioa



### — 3. PRAKTIKA

# CLUSTER HEALTH ETA MAPPING-A AZTERTU

**HELBURUA:** Elasticsearch klusterraren egoera aztertu eta **produktuak** indizearen **shard-ak** eta **mapping-a** ikusi.

## PAUSOAK:

1. Klusterraren egoera aztertu.

```
GET _cluster/health
```

- Identifikatu: **status**, **number\_of\_nodes**, **active\_primary\_shards**, **active\_shards**

2. **produktuak** indizearen shard-ak aztertu:

```
GET _cat/shards/produktuak?v
```

- Identifikatu: Primary shard-a ( **p** ), Replica ( **r** ), **STARTED** / **UNASSIGNED** egoerak

4. Indizearen mapping-a kontsultatu.

```
GET produktuak/_mapping
```

- Identifikatu Elasticsearch-ek automatikoki esleitutako datu-motak: izena, kategoria, prezioa, stock

5. Erantzun:

- a. Zein da klusterraren egoera? Zergatik?
- b. Zer gertatzen da replica-rekin? Zergatik?
- c. Zein datu-mota esleitu dio Elasticsearch-ek eremu bakoitzari?
- d. **izena** eta **kategoria** eremuetan, zergatik agertzen dira **text** eta **keyword**?

# 03

## Query DSL eta agregazioak

# Query DSL: kontsultatzeko lengoaia

**Query DSL:** JSON formatuko bilaketa-lengoaia — SQL-ren WHERE-ren baliokidea.

**Bi familia:** full-text (text eremuetan, `\_score`) eta term (keyword/zenbaki).

**Full-text:** match (OR), multi\_match (pisuak), match\_phrase (esaldia).

**Term:** term/terms (=/IN), range (tarteak, now-1h).



## Full-text vs Term

# Elasticsearch: Query DSL Kontsulta Motak

### Full-text Kontsultak

text eremuetan · tokenizatua · `_score` kalkulatu



#### match

Eremu batean hitz-bilaketa (OR/AND)



#### multi\_match

Hainbat eremutan aldi berean (pisuekin)



#### match\_phrase

Esaldi osoa, hurrenkera berean

Erabilera: testu libreko bilaketa

### Term Kontsultak

keyword/zenbaki/data · zehatza · cacheatua



#### term

Balio zehatza



#### terms

Balio-zerrenda (IN)



#### range

Tartea: gte, lte, gt, lt



#### exists

Eremu existentzia



#### wildcard/fuzzy

Patroiak eta akats-tolerantzia

Erabilera: iragazketa zehatza

### Konposatutako Kontsultak

beste kontsultak konbinatu



#### bool

must + should + must\_not + filter

- **must**: beharrezkoa + `_score`
- **should**: aukerakoa + `_score` hobetu
- **must\_not**: baztertu
- **filter**: beharrezkoa + cacheatua ⚡

Erabilera: kontsulta konplexuak

**Full-text** (`match``) — text eremuetan, analizatua, ``_score``. **Term** (`term``, `range``) — balio zehatza, ez tokenizatu.

💡 **filter > must** errendimenduarentzat — ez du `_score` kalkulatu eta cacheatzen da



# Bool Kontsulta: Lau Klausulak

## must ✓

Dokumentua **BETE BEHAR** du  
\_score-an **ERAGINA DU** ✓

```
{ "match": { "izena": "koaderno" } }
```

SQL: **"AND"**

## should 🤔



Dokumentua bete **DEZAKE**  
\_score **HOBETU** egiten du ?  
minimum\_should\_match: 1

```
{ "term": { "kategoria": "bulegoa" } }
```

SQL: **"OR"** (puntuazioa hobetu)

## bool

## must\_not ✗

Dokumentua **EZ** du bete behar  
Ez du \_score-an eraginik ✗

```
{ "term": { "eskuragarri": false } }
```

SQL: **AND NOT**

## filter ⚡



Dokumentua **BETE BEHAR** du  
Ez du \_score-an eraginik  
**CACHEATUA → AZKARRAGOA** ⚡

```
{ "range": { "prezioa": { "gte": 1 } } }
```

SQL: **AND** (optimizatua)

⚡ **Arau praktikoa:** Zenbakizko iragazketak eta datak BETI filter barruan — must baino azkarragoa

## — TRESNA NAGUSIA

## Bool kontsulta

Lau klausula: **must** (behar + score),  
**should** (aukerakoa + score),  
**must\_not** (baztertu), **filter** (behar,  
score gabe, cacheatua).

# must vs filter — errendimendua

|                  | MUST           | FILTER                       |
|------------------|----------------|------------------------------|
| Bete behar du    | Bai            | Bai                          |
| _score kalkulatu | Bai            | EZ                           |
| Cacheatua        | Ez             | Bai ⚡                        |
| Noiz             | Testu-bilaketa | Zenbaki, data, balio zehatza |



# Elasticsearch: Agregazio Motak



## Bucket Agregazioak

Dokumentuak taldetan sailkatu

**terms:** kategoriaka taldekatu

**date\_histogram:** eguneko taldeak

**range:** prezio-tarteak

→ SQL GROUP BY baliokidea



## Metrika Agregazioak

Talde barruko kalkulu numerikoak

**avg:** batezbestekoa

**max:** maximoa

**min:** minimoa

**sum:** batura

**stats:** laburpen osoa

**percentiles:** p50/p95/p99

→ SQL SUM(), AVG(), COUNT() baliokidea



## Kabiatutako Agregazioak

Bucket barruan metrika kalkulatu

**terms** (kategoria)

→ **avg** (batezbesteko prezioa)

→ **max** (prezio maximoa)

→ **value\_count** (produktu kopurua)

→ "Kategoria bakoitzeko batezbesteko prezioa"

|            |         |           |                      |
|------------|---------|-----------|----------------------|
| bulegoa:   | 45 prod | avg €3.20 | ■■■■■■■              |
| tresnak:   | 23 prod | avg €8.50 | ■■■■■■■■■■■■■■■■■■■■ |
| papeleria: | 12 prod | avg €1.80 | ■■■                  |

## — ANALITIKA

# Agregazioak: GROUP BY eta gehiago

**terms** (≈GROUP BY) ·

**date\_histogram** (denbora-serieak,  
Kibana grafikoak) · **percentiles**  
(p50/p95/p99 — latentzia).

DATO

p99 = 5.000 ms

batez bestekoa 50 ms zenean

Percentilak > batezbestekoa

API batek \*\*50 ms batez bestekoa\*\* baina \*\*p99 = 5.000 ms\*\* badu, erabiltzaileen %1ak 5 segundo itxaroten du. Batezbestekoak arazoa ezkututzen du.

# Bool kontsulta — Dev Tools

```
kontsulta.json  json

1 | GET produktuak/_search
2 | {
3 |   "query": { "bool": {
4 |     "must": [{ "match": { "izena": "koaderno"
5 |   } }],
6 |   "filter": [{ "range": { "prezioa": {
7 |     "gte": 1, "lte": 10 } } } ]
8 | }
```

## MUST

Testu-bilaketa (izena=koaderno) — \_score kalkulatzen du.

## FILTER

Prezio-tartea — cacheatua, \_score gabe, azkarra.

## DEV TOOLS

Kibana-n CURL gabe, autokonpletzioarekin.



## — 4. PRAKTIKA

# QUERY DSL: LEHEN BILAKETAK

**HELBURUA:** Query DSL erabiliz **produktuak** indizean **testu-bilaketak**, **prezio-iragazkiak** eta **baldintza konbinatuak** egitea

**PAUSOAK:**

1. Bilatu izenean **koaderno** hitza duten produktuak:

```
GET produktuak/_search
{
  "query": {
    "match": {
      "izena": "koaderno"
    }
  }
}
```

- Aztertu emaitzetako **\_score** balioa

2. Bilatu 10 € edo gutxiago balio duten produktuak:

```
GET produktuak/_search
{
  "query": {
    "range": {
      "prezioa": {
        "lte": 10
      }
    }
  }
}
```

3. Konbinatu bi baldintzak bool erabiliz:

```
GET produktuak/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "izena": "koaderno"
          }
        }
      ],
      "filter": [
        {
          "range": {
            "prezioa": {
              "gte": 1,
              "lte": 10
            }
          }
        }
      ]
    }
  }
}
```

## — 4. PRAKTIKA

# QUERY DSL: LEHEN BILAKETAK

**HELBURUA:** Query DSL erabiliz **produktuak** indizean **testu-bilaketak**, **prezio-iragazkiak** eta **baldintza konbinatuak** egitea

**PAUSOAK:**

4. Erantzun:

- a. Zein produktu aurkitzen ditu **match** kontsultak?
- b. Zein produktu aurkitzen ditu **range** kontsultak?
- c. Zein produktu geratzen dira azken **bool** kontsultan?
- d. Zer funtzio betetzen du **must**-ek? Eta **filter**-ek?

5. **Erronka:** Sortu kontsulta bat 10€ eta 50€ arteko produktuak bilatzeko.

# ARIKETA GEHIGARRIAK

1. Bilatu **20€ baino garestiagoak** diren produktuak
2. Bilatu **izena** eremuan **mekanikoa** hitza duten produktuak
3. Bilatu **5€ eta 25€ arteko** produktuak.
4. **bool** erabiliz, bilatu izenean **koaderno** duten eta **5€ baino garestiagoak** diren produktuak.
5. **Erronka**: bilatu **Informatika** kategoriako produktuak **term** erabiliz. Zer gertatzen da **kategoria** erabiltzen baduzu? Eta **kategoria.keyword** erabiltzen baduzu?

# 04 Kibana

— TRESNAK

# Kibana: ES-ren azala

Portu lehenetsita: <http://localhost:5601>

— Kibana-k beti **Elasticsearch-era kontsultak** bidalzen ditu atzean.

### Kibana: Tresna Nagusiak

#### Discover

Datu-esploratzea

- Bilaketa adierazpenak (KQL)
- Denboratarte filtratzea
- Eremu aukeraketa
- Dokumentu zuzenak ikusi

#### Visualize / Lens

Bistaratzek sortu

- Bar/Line/Pie grafikak
- Metrika eta taulak
- Drag & drop interfazea
- TSVB denb.-lerroak

#### Dashboard

Ikuspegi globala

- Bistaratzek konbinatu
- Interaktiboki iragaztea
- Auto-freskatzea
- Exportatu (PDF, PNG)

#### Maps

Mapa bistaratzea

- Kokapena-oinarritutako analisisa
- geo\_point eta geo\_shape
- Datu-geruza anitzak
- Choropleth mapak

#### Dev Tools Console

```
GET /produktuak/_search
{"query": {"match": {
  "izena": "elasticsearch"
}}}
```

Elasticsearch APIra zuzeneko sarbidea

#### Stack Management

- Index Pattern / Data View sortu
- ILM (Index Lifecycle Mgmt)
- Erabiltzaileak eta Rolak
- Snapshot Politikak

#### Alerting eta ML

- Threshold-oinarritutako alertak
- Anomalia detekzioa (ML)
- Email/Slack jakinarazpenak
- Watcher (Enterprise)

Kibana: :5601 | Elasticsearch APIrako kontsola integratua



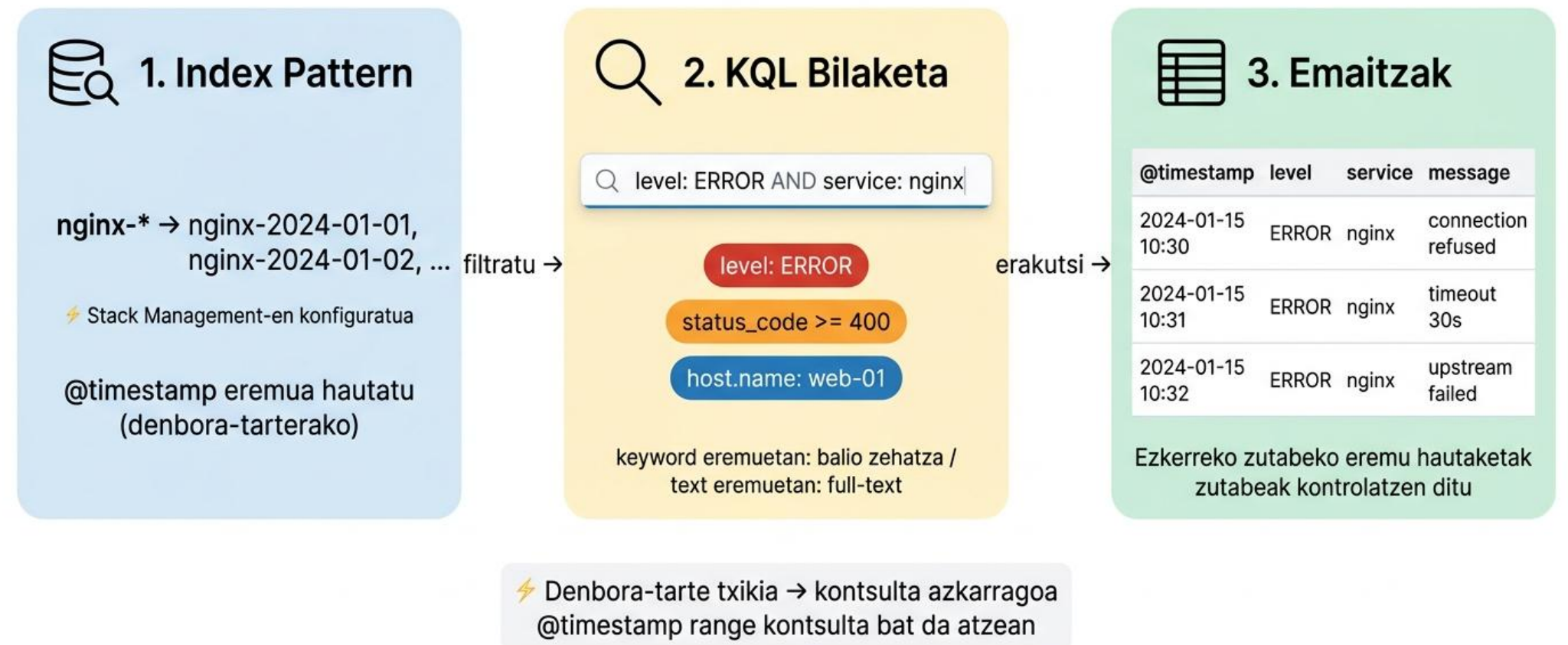
## — DISCOVER: LOGAK ESPLORATU

# Discover eta KQL

Hiru Urrats: Index Pattern  
→ KQL → Emaizak

- **Discover** — dokumentu gordinak esploratu eta iragazi
- **KQL** — bilaketa sinplifikatua: level: ERROR AND service: nginx
- **Index pattern (nginx-\*)** — hainbat indize batera
- **Denbora-tartea:** Last 15 min, Last 7 days...

## Kibana Discover: Nola Funtzionatzen Du



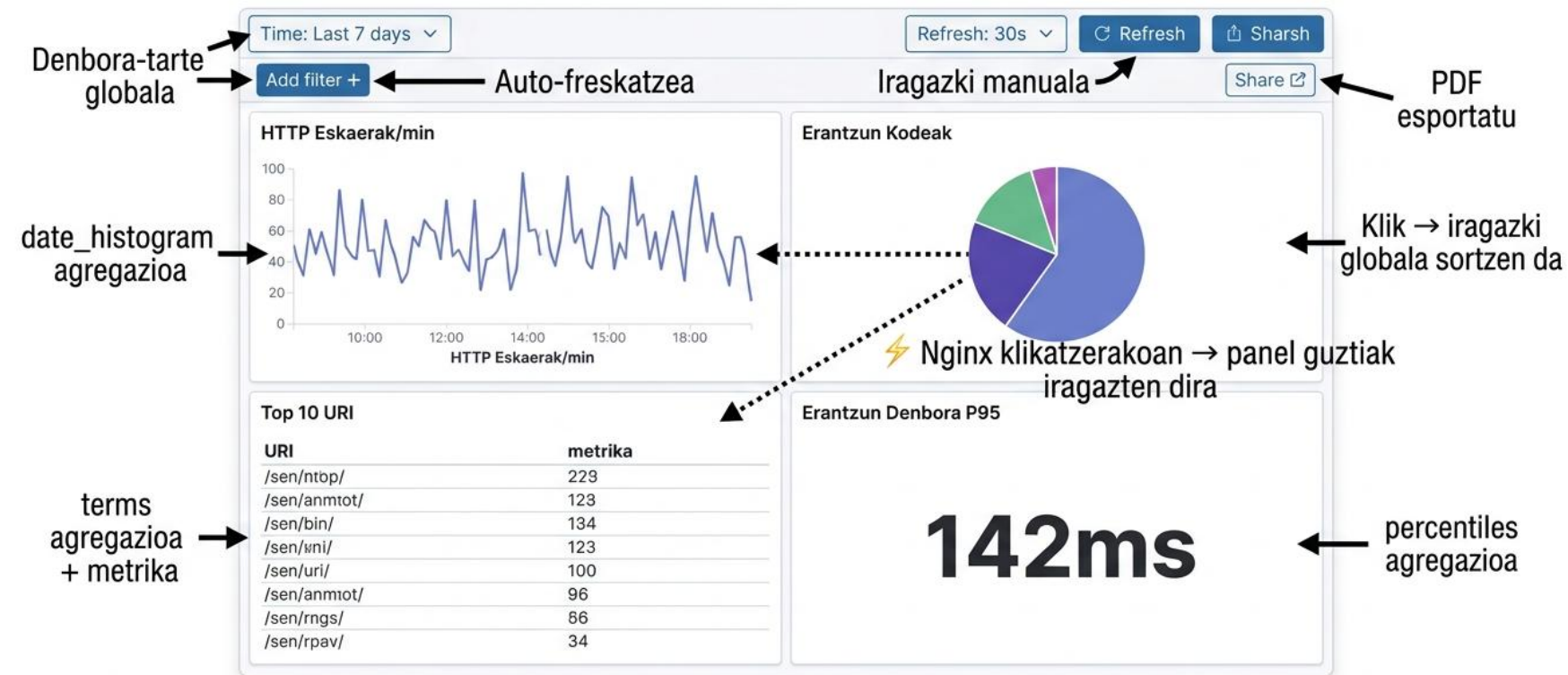
## — DASHBOARD: BISTARATZEA

# Dashboard baten anatomia

## — Panel nagusiak

- **Panelak konbinatu** — ikuspegi orokorra istorio bat
- Denbora-tartea panel GUZTIEI batera aplikatzen zaie
- Panel batean klik → guztiak iragazten dira
- **Auto-freskatzea (30s)** — monitorizazio errealean

### Kibana Dashboard: Anatomia



**Atzean:** panel bakoitza Elasticsearch kontsulta independente bat da — baina denbora-tarte globala guztiei aplikatzen zaie



# ILM — Index Lifecycle Management



## — STACK MANAGEMENT

### ILM — datuen temperatura

**Hot** (7 egun, SSD) → **Warm** (30 egun, HDD) → **Cold** (90 egun) → **Delete** (180 egun). 1. moduluko datuen temperatura praktikan.

ILM estrategiaren onura nagusia: Hot faseko disko garestia soilik azken datuentzat erabiliz, kostua nabarmen murrizten da.

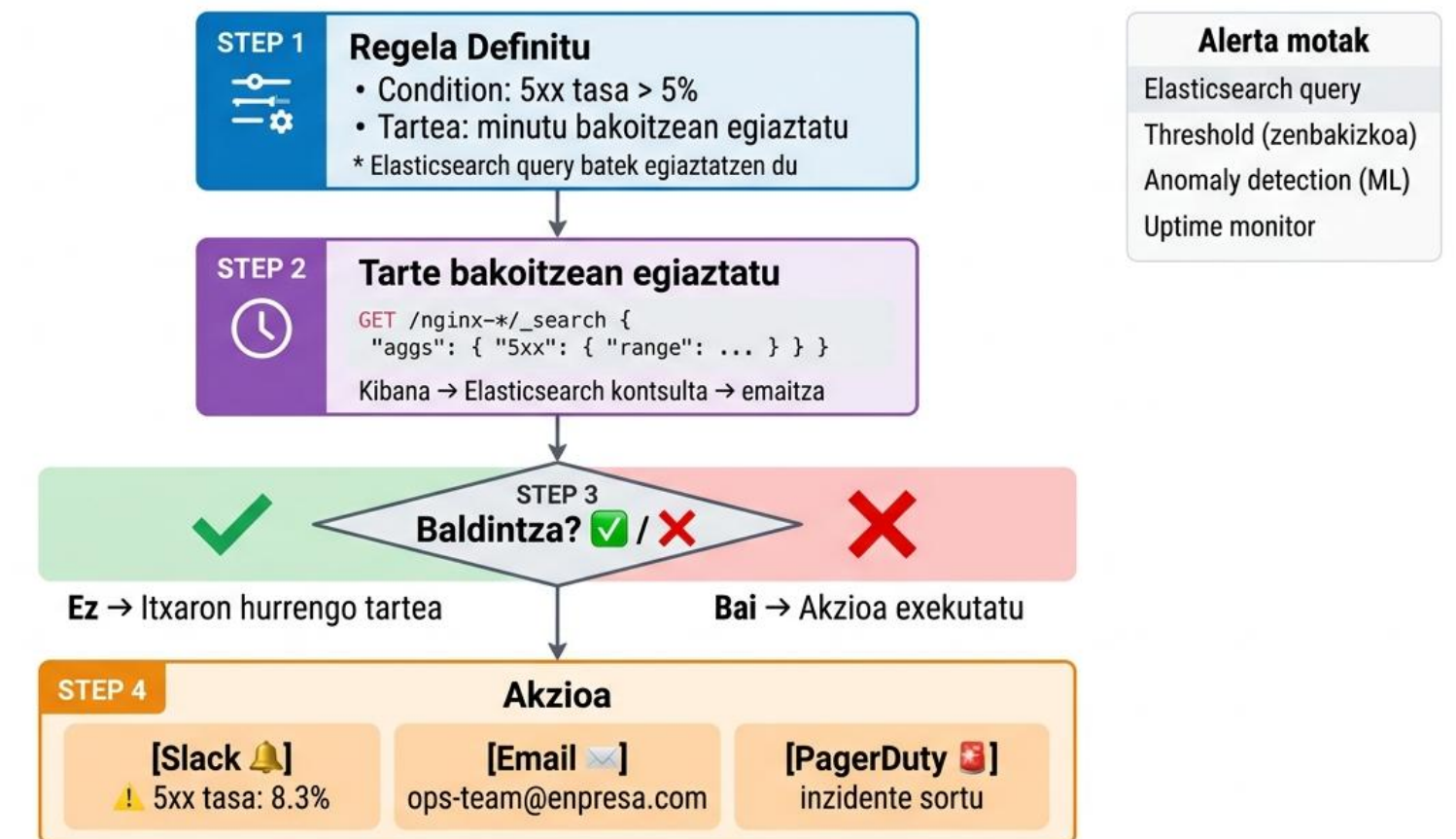
## — ALERTING: MONITORIZAZIOA

# Alerting: Jakinarazpen automatikoak

Kibana-k atzean Elasticsearch-era kontsulta bidalzen du tarteka — baldintza betetzen bada jakinarazpena bidalzen du.

- Atalase bat definitu (adib. 5xx tasa > %5)
- Tarteka egiaztatu (adib. minutu bakoitzean)
- Akzioa: Slack, email, PagerDuty
- ML: anomalia-detekzioa atalase manual gabe

## Kibana Alerting: Nola Funtzionatzen Du



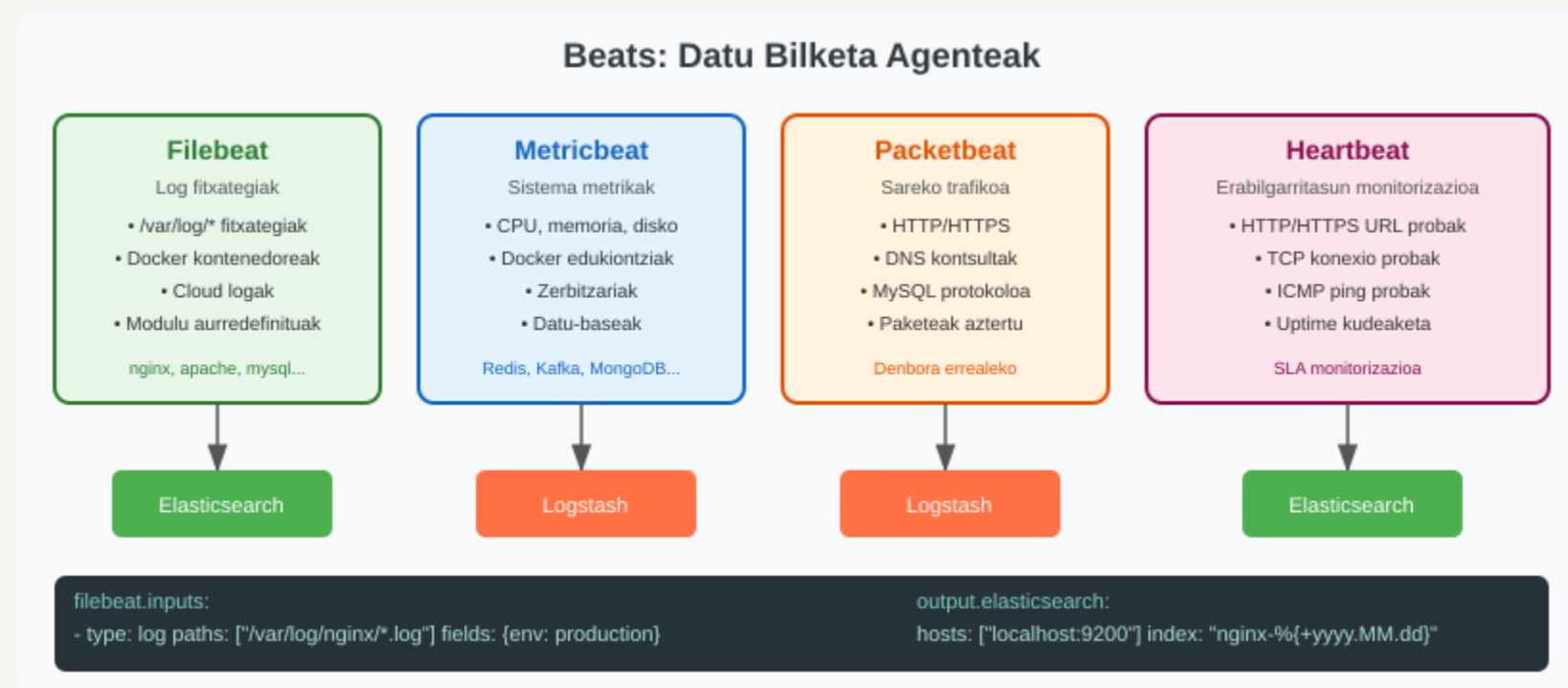
05

# Ingesta: Beats eta Logstash



## — AGENTE ARINAK

# Beats



- Iturrian instalatutako agente arinak (~50 MB RAM)
- Filebeat (logak), Metricbeat (CPU/RAM), Packetbeat (sarea)
- Heartbeat (uptime), Winlogbeat, Auditbeat
- Datuak Logstash-era edo zuzenean ES-era bidaltzen dituzte

# Logstash: testu gordina → eremu egituratuak

**Arazoa:** `192.168.1.105 ... "GET /api" 200 1234` — testu gordina. Ezin duzu `status\_code: 500` bilatu.

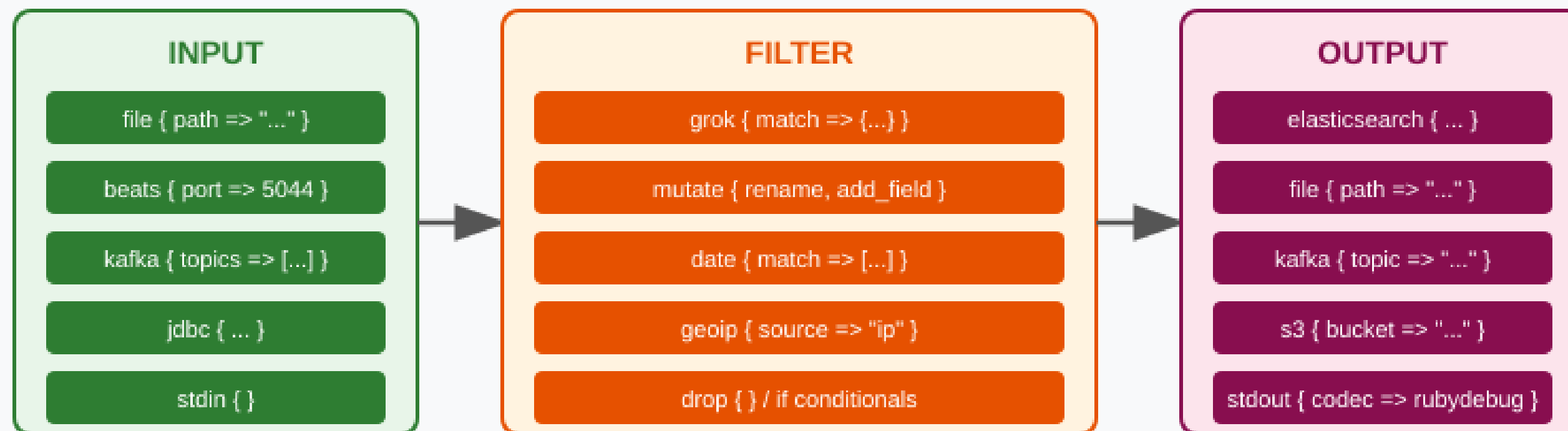
**Datua existitzen da, baina ITXITA dago.**

**Logstash:** testu gordina hartu, eremutan banatu, JSON egituratu bihurtu.

Hiru fase: **input** → **filter** → **output** (jaso → eraldatu → bidali).

## — PIPELINE

Input → Filter → Output



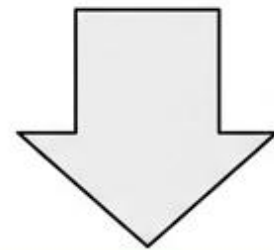
Logstash pipeline: datuak jaso → eraldatu → helmugara bidali

**Input** (Beats, Kafka, fitxategiak) ·  
**Filter** (grok, date, geoip, mutate) ·  
**Output** (Elasticsearch, S3, Kafka).

## grok: Testu Gordina → Eremu Egituratura

```
192.168.1.105 - - [15/Jan/2024:10:30:45 +0100] "GET /api/v1 HTTP/1.1" 200 1234
```

```
%{IP:bezero_ip} %{WORD:metodoa} %{URIPATH:request} HTTP/%{NUMBER:httpversion}  
%{NUMBER:status_code:int} %{NUMBER:bytes:int}
```



**grok filter**

```
{  
  "bezero_ip": "192.168.1.105",  
  "metodoa": "GET",  
  "request": "/api/v1",  
  "httpversion": "1.1",  
  "status_code": 200,  
  "bytes": 1234  
}
```

**%{IP}** → IPv4/IPv6 helbidea

**%{WORD}** → Hitz bat

**%{URIPATH}** → URL path

**%{NUMBER}** → Zenbakia

**%{GREEDYDATA}** → Edozer (⚠ astiro)

💡 **Kibana Grok Debugger:**  
Dev Tools → Grok Debugger.  
Pattern garatu aurretik probatu!

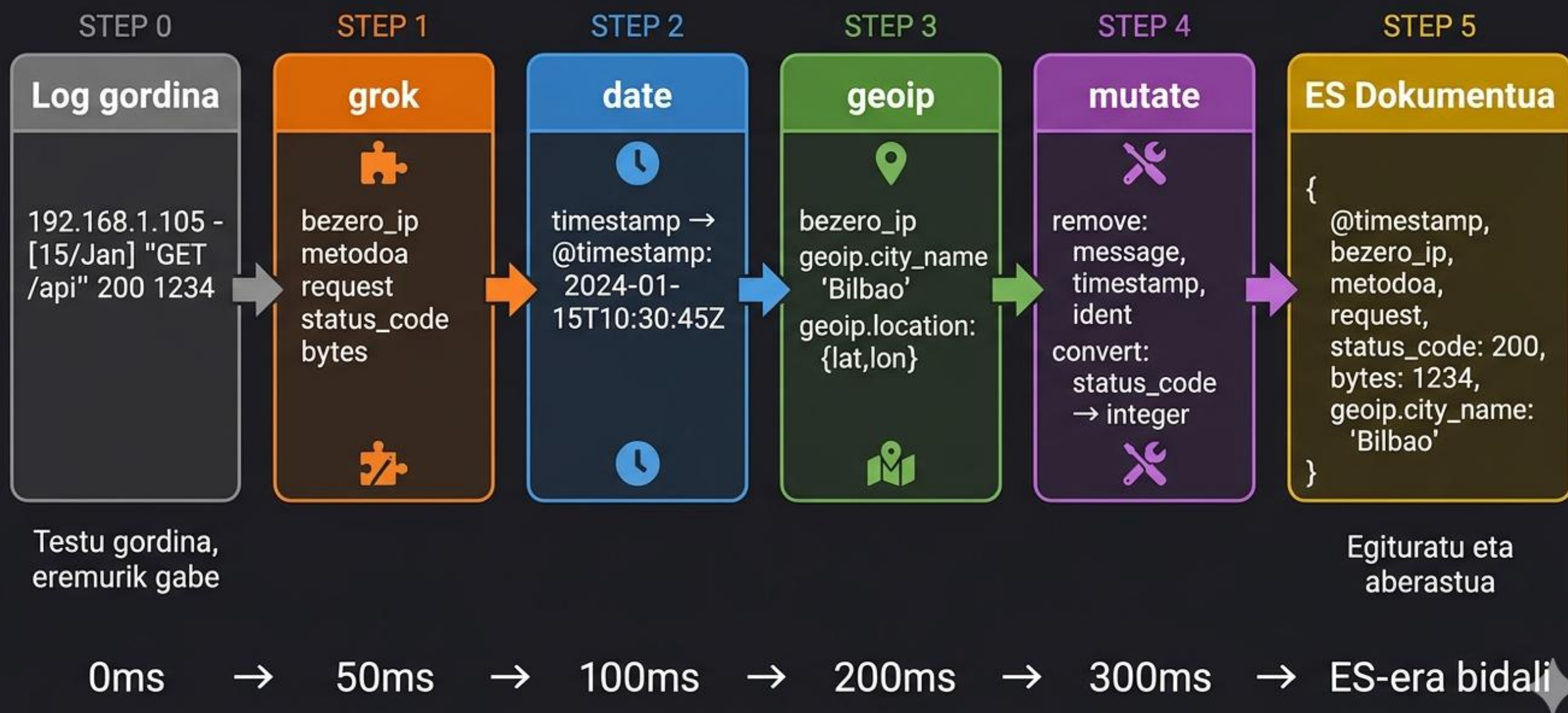
— FILTER GAKOA

## grok — testu gordina eremu bihurtu

Patroi izendunak: ``%{IP:bezero_ip}`` →  
IP helbidea hartu eta eremu batean  
gorde. Nginx/Apache:  
``%{COMBINEDAPACHELOG}`` → 10+  
eremu berehala.



# Nginx Log → Filter Katea → ES Dokumentua



## — FILTER KATEA

grok → date →  
geoip → mutate

Log-lerro baten bizi-zikloa: parseatu  
(grok) → timestamp (date) →  
kokapena (geoip) → garbitu (mutate)  
→ ES dokumentu aberastua.



# Filter pitfall klasikoa

## *date filter ahaztea*

Log-ak bere ordua darama, baina ES-ek @timestamp espero du. date filter GABE, @timestamp = Logstash-ek prozesatu duen unea, ez log-aren benetako data → grafikoetan desplazamendua.

### IMPORTANTE

Beti jarri date filter-a. Bestela monitorizazio-grafikoetan logak okerreko unean agertzen dira (prozesatu zirenean, ez gertatu zirenean).

# Logstash vs Ingest Node

- **Logstash** — grok konplexua, iturri anitz, logika baldintzatua
- **Ingest Node** — ES barnean, eraldaketa sinpleak, nodo gabe
- Erregela: log gordina parseatu → Logstash
- JSON jada egituratua + eraldaketa txikia → Ingest Node

## Logstash vs Ingest Node: Noiz Aukeratu?

|  <b>Logstash</b><br>"Pipeline indartsu eta malgua"                                                                                                                                                                       |  <b>Ingest Node</b><br>"Sinplea, ES barnean integratua"                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>✓ Grok parsing konplexua (nginx, apache, syslog)</li><li>✓ Iturri anizkoitz (Beats + JDBC + Kafka)</li><li>✓ GeoIP, eraldaketa aurreratua</li><li>✓ If/else logika baldintzatua</li><li>✓ 200+ plugin ekosistema</li><li>✓ Persistent Queue (datu galera 0)</li></ul> | <ul style="list-style-type: none"><li>✓ JSON logak (jada egituratuta)</li><li>✓ Eraldaketa sinpleak (eremu gehitu/kendu)</li><li>✓ ES baliabide berdinak erabili</li><li>✓ Konfigurazio gutxiago</li><li>✓ Doakoa (ES-en barnean)</li></ul> |
|  1-4GB RAM gehigarria                                                                                                                                                                                                  |  Plugin gutxiago, iturri bakarra                                                                                                                       |
| <b>Arau praktikoa:</b> Log gordina parseatu behar? → <b>Logstash</b> . JSON jada egituratua? → <b>Ingest Node</b> .<br>Iturri anizkoitz? → <b>Logstash</b> . Eraldaketa sinplea? → <b>Ingest Node</b>                                                                                                       |                                                                                                                                                                                                                                             |

# Gogoan hartzekoak

- 
- i* **Alderantzizko indizea** — zergatik azkarragoa SQL LIKE baino

---

  - ii* **text vs keyword** — term text eremuan ez du ezer aurkitzen

---

  - iii* **bool**: must (score) vs **filter** (cacheatua, iragazketarako)

---

  - iv* **Agregazioak**: terms (GROUP BY), date\_histogram, percentiles (p99)

---

  - v* **Logstash**: grok (eremuak) + date (@timestamp ezinbestekoa)

ESKERRAK

# Galderak eta eztabaida.

MODULUA

3 · Elastic Stack